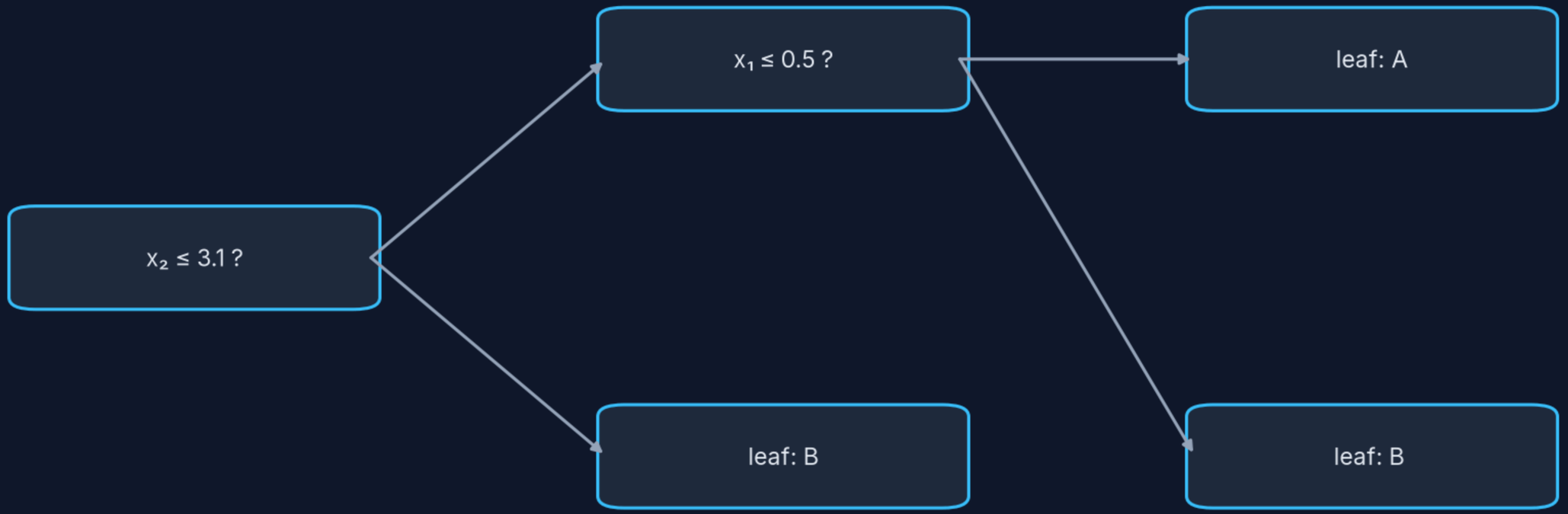
Decision Trees

CART and Random Forests

The problem

- Non-linear boundaries, mixed feature types
- Want rules a human can read
- No feature scaling required

Intuition: twenty questions



Choosing a split: Gini impurity

$$G = 1 - \sum_k p_k^2$$

$$G_{split} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R$$

- Try every feature and threshold
- Keep the split with the lowest weighted impurity

Greedy split search

```
def gini(y):
    _, c = np.unique(y, return_counts=True)
    p = c / c.sum()
    return 1 - np.sum(p * p)

# inside DecisionTree._grow
for f in feats:
    for t in np.unique(X[:, f])[:-1]:
        left = X[:, f] <= t
        g = (left.sum() * gini(y[left])
             + (~left).sum() * gini(y[~left])) / len(y)
        if best is None or g < best[0]:
            best = (g, f, t)
```

Random forest

- Bootstrap: each tree sees a resample of the data
- Each split considers $\sqrt{d}$ random features
- Predict by majority vote across trees
- Averaging de-correlated trees cuts variance

RandomForest.fit

```
def fit(self, X, y):
    X, y = np.asarray(X, float), np.asarray(y)
    rng = np.random.default_rng(self.seed)
    mf = self.max_features or max(1, int(np.sqrt(X.shape[1])))
    self.trees = []
    for i in range(self.n_trees):
        idx = rng.integers(0, len(X), len(X))
        self.trees.append(DecisionTree(self.max_depth, max_features=mf, seed=self.seed + i).fit(X[idx], y[idx]))
    return self
```

Complexity & trade-offs

- Naive split search: $O(d \cdot n^2)$ per node; sort-based $O(d \cdot n \log n)$
- Prediction: O(depth)
- Single tree: interpretable but high variance
- Forest: accurate, robust, less interpretable

Recap

- Trees ask greedy threshold questions
- Split by minimum weighted Gini
- Limit depth to control overfitting
- Forests = bagging + feature subsampling